```
class CVModelWrapper(mlflow.pyfunc.PythonModel):
    def __init__(self, model):
        # instantiate model in evaluation mode
        model.to(torch.device("cpu"))
        self.model = model.eval()

    def feature_extractor(self, image, p=0.5):…

    def predict(self, context, images):
        id2label = {
            0: 'buildings',
            1: 'forest',
            2: 'glacier',
            3: 'mountain',
            4: 'sea',
            5: 'street'
            }
        with torch.set_grad_enabled(False):

          # assuming this is a DataFrame
          pil_images = torch.stack([self.feature_extractor(row[0]) for _, row in images.iterrows()])
          pil_images = pil_images.to(torch.device("gpu"))
          outputs = self.model(pil_images)
          preds = torch.max(outputs, 1)[1]
          probs = torch.nn.functional.softmax(outputs, dim=-1)[:, 1]
          labels = [id2label[pred] for pred in preds.tolist()]

          return pd.DataFrame(
                                data=dict(
                                  label=preds,
                                  labelName=labels
                                  )
                              )
```

Figure 7.31 – Including feature_extractor and predict functions in the model class wrapper

Now, we are ready to wrap our fine-tuned model from *Chapter 4* into the wrapper. To do this, we load the artifact back from MLflow and pass it to the pre-created class `CVModelWrapper` above.